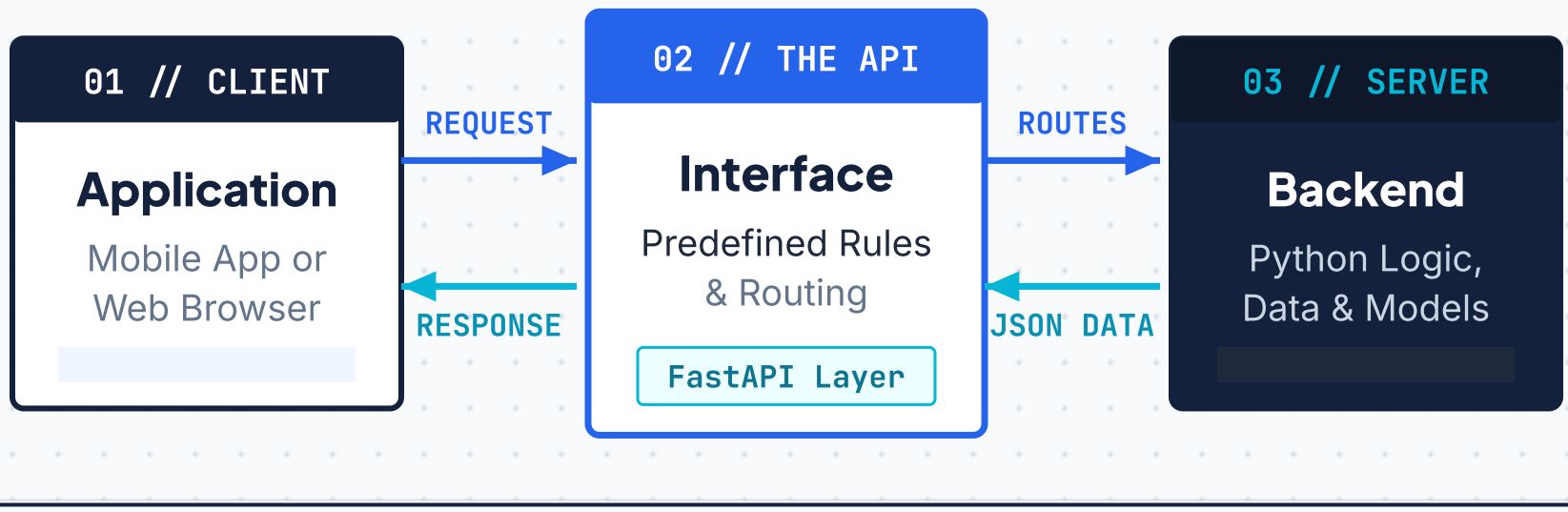
BACKEND ARCHITECTURE SERIES // MODULE 01

FASTAPI: UNDERSTANDING THE BASICS

> “Before building APIs, understand what happens behind the scenes.”

FIG 01.0 // CORE SYSTEM COMMUNICATION TOPOLOGY

HTTP / JSON PROTOCOL



PREREQUISITE	BACKEND EXP	FOCUS
Basic Python	**None Required**	**Core API Concepts**

01 // THE COMMUNICATION GAP

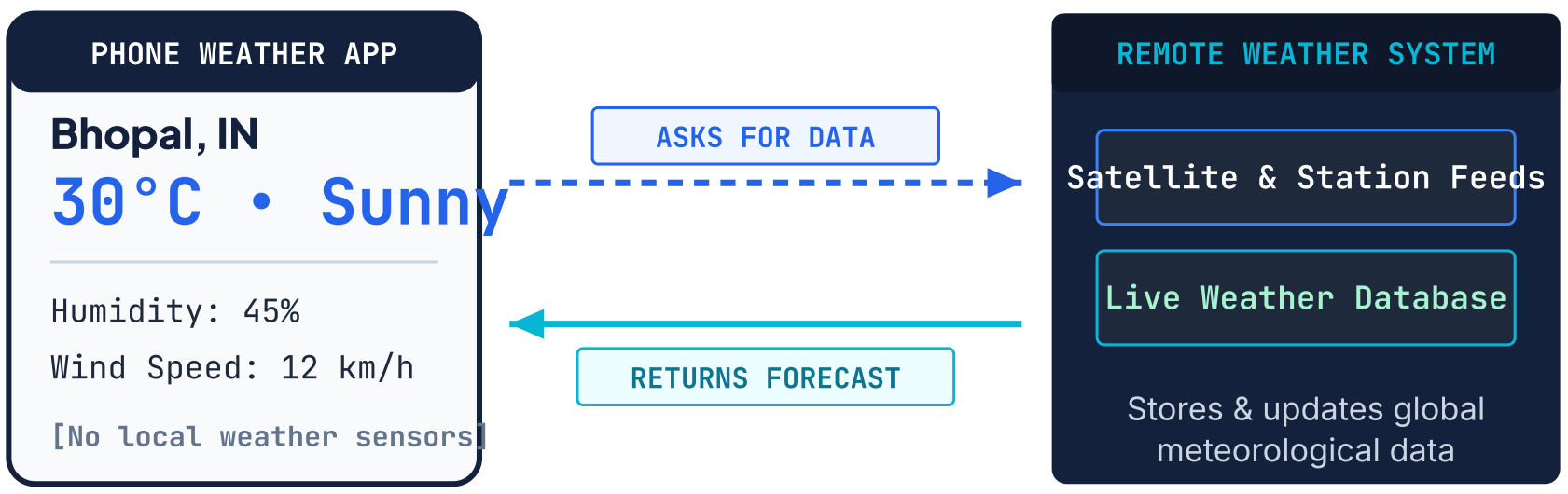
How do two different applications communicate?

Imagine you open a weather application on your phone. Instantly, you see four pieces of live information:

METRIC 01	METRIC 02	METRIC 03	METRIC 04
Current Temperature	**Weather Conditions**	**Relative Humidity**	**Current Wind Speed**

But where does this information come from?

Your phone does not independently know the current weather. It needs to communicate with another system that has this information.



This is where APIs come in.

NEXT: DEFINING THE API →

02 // CORE CONCEPT

API = Application Programming Interface

An **API** is a way for two software applications to communicate with each other through **predefined rules**.

MENTAL MODEL // THE RESTAURANT ANALOGY

01. CUSTOMER	02. WAITER	03. KITCHEN	04. FOOD
Client	**API**	**Server**	**Response**
Reads the menu and places an order.	Carries the order and returns the food.	Prepares the dish behind the scenes.	The requested data delivered back.

ROUND-TRIP FLOW: HOW THE MESSENGER OPERATES

CUSTOMER → WAITER → KITCHEN → WAITER → CUSTOMER



IMPORTANT TECHNICAL DISTINCTION

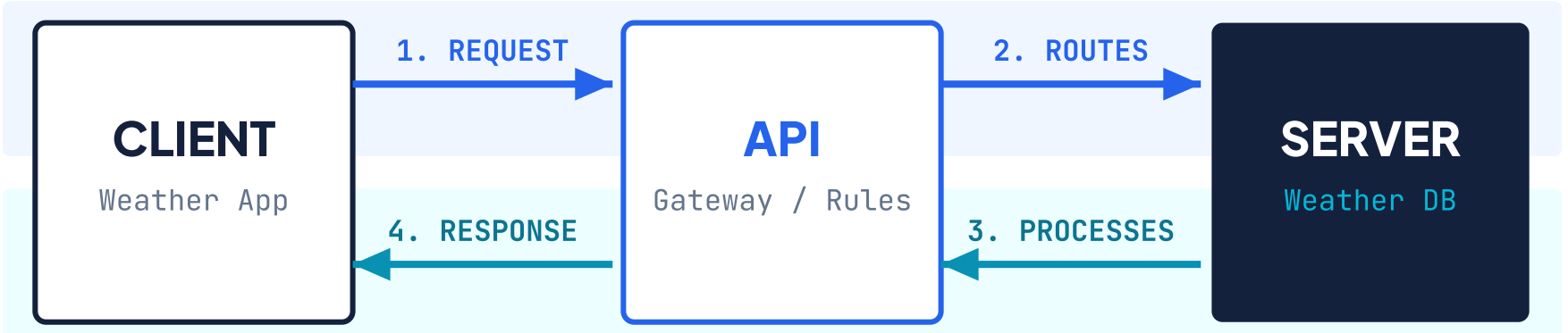
This restaurant model is an **analogy** to help visualize the flow. A real API is not a person—it is a strict software contract with predefined rules for exchanging data.

03 // STEP-BY-STEP EXECUTION

Let's understand the communication process.

FIG 04.1 // WEATHER APP REQUEST-RESPONSE CYCLE

RUNNING EXAMPLE: BHOPAL WEATHER



01 **Client sends a request**
Your phone's weather app asks for the current weather in Bhopal.

02 **API receives and routes the request**
The API verifies the request format and directs it to the weather lookup endpoint.

03 **Server processes the request**
The backend server queries its database and retrieves Bhopal's latest temperature and humidity.

04 **Server sends a response back**
The server returns the structured weather data back to your phone for display.

04 // ARCHITECTURAL ROLES

Who is talking to whom?

CLIENT

INITIATOR

The application or device that **requests** information or a service.

COMMON EXAMPLES:

01. Mobile application
02. Web browser
03. Frontend application

SERVER

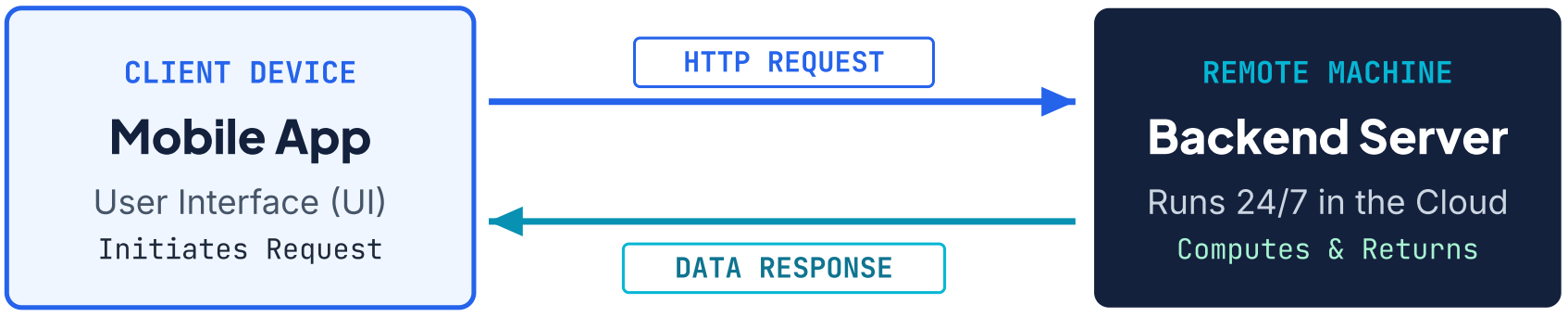
PROVIDER

The system that **receives** requests, performs processing, and **returns** responses.

COMMON EXAMPLES:

01. Weather data server
02. Banking backend
03. E-commerce backend

FIG 05.1 // DIRECT CLIENT-SERVER INTERACTION ACROSS THE NETWORK



05 // THE API CONVERSATION

Every API conversation has two important parts.

PART 01 // REQUEST

What the client sends to the server.

PART 02 // RESPONSE

What the server sends back.

MESSAGE PANEL 01 // REQUEST (CLIENT → SERVER) OUTBOUND →

CONCEPTUAL MEANING OF THE REQUEST:

> “Give me the weather information for Bhopal.”

← INBOUND MESSAGE PANEL 02 // RESPONSE (SERVER → CLIENT)

RETURNED WEATHER DATA PAYLOAD:

Temperature	Condition	Humidity
30°C	Sunny	45%

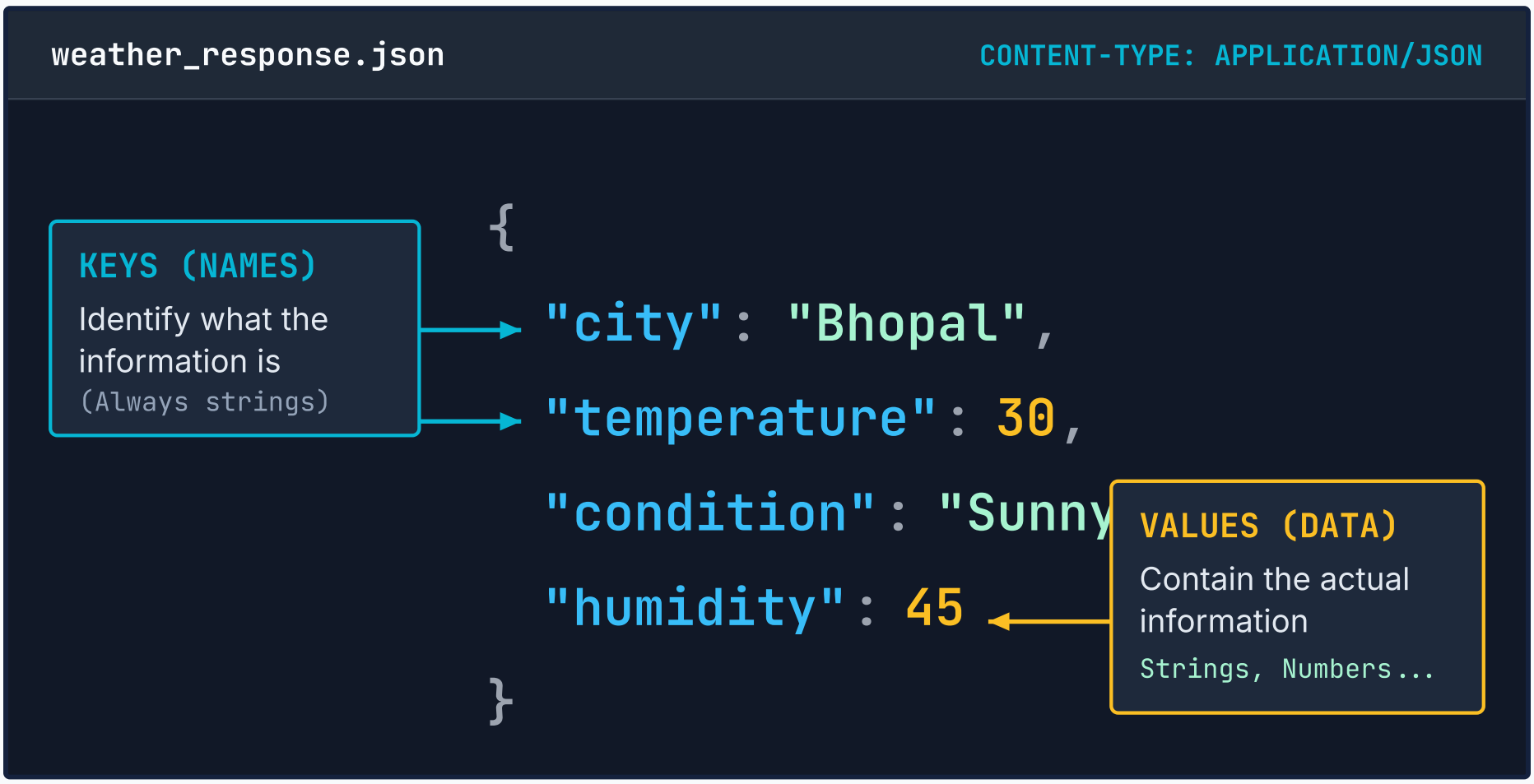
HOW REAL COMPUTERS FORMAT MESSAGES

Actual API communication uses **structured data** rather than ordinary conversational sentences—which brings us to **JSON**.

06 // STRUCTURED DATA EXCHANGE

How does an API send structured data?

APIs commonly use **JSON (JavaScript Object Notation)**—a lightweight format for exchanging data between applications.



01 // KEYS

Identify each field (such as `"city"` or `"temperature"`).

02 // VALUES

Hold the actual data (`"Bhopal"`, `30`, `"Sunny"`).

03 // PYTHON FIT

Looks almost identical to a standard Python `dict`!

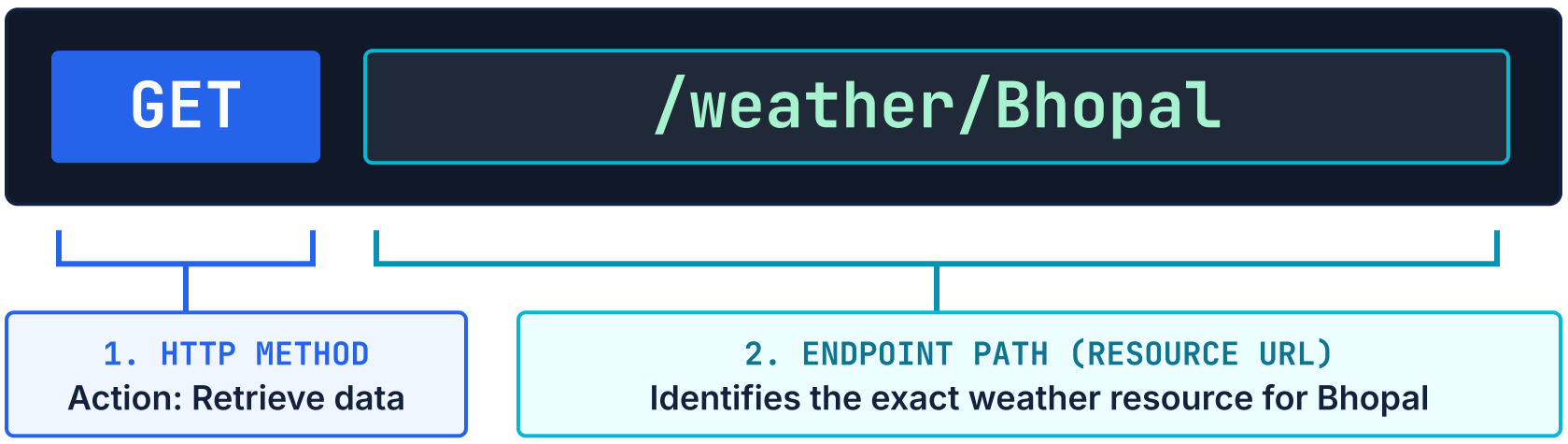
07 // ARCHITECTURAL STYLE

You will often hear the term REST API.

REST is an architectural style for designing network-based APIs. REST APIs commonly use **HTTP** to allow clients to interact with resources (pieces of data).

ANATOMY OF A REST API REQUEST

HTTP + RESOURCE ENDPOINT



`GET` **The HTTP method:** Tells the server we want to read or retrieve information.

`/weather/Bhopal` **The endpoint path:** Points to the specific resource being requested.

SERIES ROADMAP NOTE

Together, this request asks for weather information about Bhopal. We will explore all HTTP methods (`GET` , `POST` , `PUT` , `DELETE`) in a later part.

08 // CONNECTING TO PYTHON

Now, let's connect this to Python.

FastAPI is a modern Python web framework used to build APIs.

WHAT FASTAPI ALLOWS PYTHON DEVELOPERS TO DO:

01. Receive incoming client requests

02. Process data using Python

03. Validate input automatically

04. Return structured JSON responses

05. Connect machine learning models to web & mobile applications

FIG 09.1 // END-TO-END FASTAPI WORKFLOW




```python
# PREVIEW: OUR WEATHER ENDPOINT IN FASTAPI (SERIES FRAMEWORK)
@app.get("/weather/Bhopal")
def get_weather():
    return {"city": "Bhopal", "temperature": 30}
```

09 // QUICK REFERENCE MATRIX

Before moving forward, remember these concepts.

CONCEPT	CORE MEANING
API	Communication interface between applications
Client	Sends a request
Server	Processes the request
Request	Information sent to the server
Response	Information returned by the server
JSON	Structured data format
REST API	API designed around REST principles
FastAPI	**Python framework for building APIs**

COMING NEXT // PART 02

Introduction to FastAPI — Why Python Developers Use It

Save this carousel if you're learning backend development with Python.